ΜΕΘΟΔΟΙ - METHODS

ΣΤΗΝ C++

ΕΙΣΑΓΩΓΗ – ΕΠΑΝΑΛΗΨΗ ΑΠΟ ΤΑ ΠΡΟΗΓΟΥΜΕΝΑ

- με τον όρο **μέθοδο (method)** ονομάζουμε ένα αυτόνομο κομμάτι κώδικα (συνώνυμα: υπορουτίνα, συνάρτηση).
- στις περισσότερες γλώσσες αντικειμενοστραφούς προγραμματισμού μπορούμε να αναγκάσουμε μία κλάση να έχει κάποιες μεθόδους.
- Η κάθε κλάση περιέχει μηδέν ή περισσότερες **μεθόδους (methods)**.
- Αυτό γίνεται ορίζοντας μία **διεπαφή (interface)** η οποία περιέχει τις υπογραφές των μεθόδων που επιθυμούμε η κλάση να έχει.
- Λέμε ότι μία κλάση **υλοποιεί (implements)** μία διεπαφή όταν **υλοποιεί (δηλαδή προσθέτει σώμα)** σε όλες τις μεθόδους της διεπαφής.
- Αν μία κλάση υλοποιεί μία διεπαφή, τότε είναι **υποχρεωτική** η προσθήκη σώματος σε όλες τις μεθόδους της διεπαφής.

- Μέσα σε μία μέθοδο οι εντολές εκτελούνται **σειριακά**.

- Οι μέθοδοι όμως μέσα σε ένα αρχείο δεν είναι απαραίτητο ότι θα εκτελεστούν σειριακά.

- Μία μέθοδος μπορεί να καλέσει μία άλλη μέθοδο, όπου αυτή και να βρίσκεται (στο ίδιο αρχείο, σε αρχείο του ίδιου πακέτου αλλά σε αρχείο διαφορετικού πακέτου).

- Κάθε μέθοδος αποτελείται από δύο μέρη: την **υπογραφή της (method signature)** και το **σώμα της (method body)**, το οποίο περιέχει τις εντολές της. Θα αναφερθούμε στις έννοιες αυτές σε παρακάτω μαθήματα.

- Πώς όμως γνωρίζει από ποιά μέθοδο θα αρχίσει η εκτέλεση του προγράμματος;

- Σε κάποιο από τα αρχεία του προγράμματος είναι απαραίτητο να υπάρχει μέθοδος με την παρακάτω υπογραφή:

```
public static void main(String[] args)
{
    ...
}
```

- Η μέθοδος main είναι υποχρεωτικό να ανήκει σε κάποιο από τα αρχεία του προγράμματος. Σε διαφορετική περίπτωση το πρόγραμμα δεν μεταγλωττίζεται.

ΠΕΡΙΣΣΟΤΕΡΑ ΓΙΑ ΤΙΣ ΜΕΘΟΔΟΥΣ

- Μία μέθοδος είναι ένα **κομμάτι κώδικα (εντολές)** η οποία **όταν εκτελεστεί**, επιτελεί μία συγκεκριμένη λειτουργία.
- Οι μέθοδοι ορίζονται πάντα μέσα στα **σώματα κλάσεων (class)** ή **διεπαφών (interface)**.
- Κάθε μέθοδος αποτελείται από δύο μέρη: την **υπογραφή της (signature)** και το **σώμα της (body)**.
- Η γενική μορφή μίας μεθόδου είναι η ακόλουθη:

  τροποποιητής επιστρεφόμενος_τύπος όνομα(τύπος_ορίσματος1 όρισμα1, ...)
  {
  // σώμα της μεθόδου
  }

- Ο πλέον συνηθέστερος τροποποιητής είναι ο τροποποιητής **public**.
- Μπροστά από τον επιστρεφόμενο τύπο μπορούν να μπουν οι λέξεις **static**, **abstract** και **final** (είναι επιτρεπτοί και μερικοί συνδυασμοί τους).
- **Για να μπορέσει να εκτελεστεί μία μέθοδος θα πρέπει μία άλλη μέθοδος να την καλέσει (η μοναδική μέθοδος η οποία καλείται αρχικά είναι η μέθοδος main).**

- Μία μέθοδος, εκτός κι αν είναι κατασκευάστρια, **μπορεί μετά την εκτέλεσή της να επιστρέψει μία τιμή στο σημείο όπου καλέσθηκε**.

- Σε κάθε μέθοδο, εκτός κι αν είναι κατασκευάστρια, είναι **υποχρεωτικό να δηλώνει τον τύπο του δεδομένου που επιστρέφει** μετά την εκτέλεσή της. Σε περίπτωση που η μέθοδος δεν επιστρέφει κάποια τιμή μετά την εκτέλεσή της χρησιμοποιούμε τον λέξη **void**.

- Μία μέθοδος μπορεί να έχει κανένα ή παραπάνω ορίσματα σαν είσοδο. Τα ορίσματα είναι μεταβλητές, οι τιμές των οποίων κάθε φορά δίνονται στο σημείο όπου η μέθοδος καλείται.

- Για κάθε όρισμα θα πρέπει να δηλώνουμε τον τύπο του. Τα ορίσματα μεταξύ τους χωρίζονται με **κόμμα** ,.

- Αν μία μέθοδος έχει έναν οποιοδήποτε επιστρεφόμενο τύπο εκτός από void, τότε είναι υποχρεωτικό να επιστρέφει μία τιμή αυτού του τύπου σε κάθε μονοπάτι εκτέλεσής της.

- Η επιστροφή μιας τιμής γίνεται με την χρήση της εντολής return:

  return τιμή;

- Στην θέση της τιμής μπορεί να μπει επίσης και μία μεταβλητή του εν λόγω τύπου.

- Η εντολή return έχει ως αποτέλεσμα η εκτέλεση της μεθόδου να σταματά αμέσως στο σημείο αυτό και ο κώδικας επιστρέφει πίσω στο σημείο που η μέθοδος καλέσθηκε.

- Σε μία μέθοδο void μπορούμε να σταματήσουμε την εκτέλεσή της χρησιμοποιώντας την εντολή return ως εξής:

  return;

ΥΠΕΡΦΟΡΤΩΣΗ – OVERLOADING ΜΕΘΟΔΩΝ

- Στο σώμα της ίδιας κλάσης (ή της ίδιας διεπαφής) μπορούν να υπάρξουν πολλές μέθοδοι οι οποίες έχουν το ίδιο όνομα αλλά διαφορετικά ορίσματα.

- Η δυνατότητα αυτή ονομάζεται υπερφόρτωση μεθόδων (method overloading).

- Το ποια μέθοδος θα καλεστεί κάθε φορά που πραγματοποιείται κλήση μια εξ'αυτών με το ίδιο όνομα εξαρτάται από τα ορίσματα τα οποία στέλνονται.

```
public class Car
{
     ...
     public void speedUp() { speed += 10; }
     ...
     public void speedUp(int kms) { speed += kms; }
     ...
}

public static void main(String[] args)
{
    Car c1 = new Car();
    c1.speedUp();    // Καλείται η πρώτη έκδοση της speedUp
    c1.speedUp(15); // Καλείται η δεύτερη έκδοση της speedUp
}
```